

Improved Personalized Typing Predictions via Long Left Context

Soufia Bahmani and Keith Vertanen

Department of Computer Science
Michigan Technological University, Houghton, MI 49931, USA
{sbahmani, vertanen}@mtu.edu

Abstract

We study how to improve predictive typing via per-user personalization of a pretrained large language model (LLM). We evaluate personalization in a time-ordered setting, using each user’s earlier messages for adaptation as we try and predict their next message. We compare: (1) personalization versus a non-personalized base model, (2) differing amounts of left context, (3) one-time adaptation versus making small periodic updates, and (4) full fine-tuning versus parameter-efficient fine-tuning. We find both static and personalized LLMs significantly outperform an n-gram language model. This includes when the n-gram was mixed with a user model learned via prediction by partial matching (PPM). Using long recent user-specific context provided the largest personalization gains: conditioning on up to 1024 previous characters reduced the base model’s perplexity by 18%. We found fine-tuning model weights provided only a small additional gain.

1 Introduction

Typing can be slow and effortful for some people, including those with motor impairments (Cai et al., 2024), those facing situational constraints (e.g., typing on a mobile device while in motion), or those writing in a language they are learning (Sarsenbayeva et al., 2017; Haake et al., 2024). Within this space, Augmentative and Alternative Communication (AAC) provides a high-impact motivating case. AAC users often write letter-by-letter using alternative input methods such as eye tracking or single-switch (Beukelman and Light, 2020). Typical AAC text entry rates are 5–10 words per minute, an order of magnitude slower than spoken speech (Trnka and McCoy, 2008).

Text entry interfaces often try to reduce this burden by using a language model to autocorrect noisy input and by offering text predictions. However, a language model is usually trained once on large

generic corpora and then deployed unchanged for every user. This mismatch can be problematic for text entry as users may want to write specific text (e.g., names, contacts, or habitual phrases). Further, the user’s language usage can drift over time.

Previous text entry research has shown the value of personalization and using large language models (LLMs) (Adhikary and Vertanen, 2023; Gaines and Vertanen, 2025). We build on this work by studying progressive per-user LLM personalization under a chronological predict-then-update protocol.

We study progressive personalization of LLMs on real email data from the Enron corpus (Klimt and Yang, 2004). We ask how much text prediction improves when we adapt an LLM to an employee’s past messages. To measure prediction accuracy and typing effort, we use per-character perplexity and keystroke savings. Keystroke savings measures the percentage reduction in key presses compared to letter-by-letter typing in a text entry interface that suggests the next word (Trnka and McCoy, 2008).

We consider two personalization settings. In *one-time personalization*, we first adapt the model on the first half of the sent emails of a user (an Enron employee). We leave the model frozen while evaluating on the user’s second half of sent emails. In *progressive personalization*, we periodically update the model as we pass through the second half of the emails. Progressive personalization aims to adapt to ongoing changes in a user’s writing.

We make the following contributions:

1. We show an LLM without personalization outperforms a strong statistical baseline model adapted to a user’s writing.
2. We show that using large amounts of left context provides effective personalization without the need for updating the weights of an LLM.
3. We find parameter-efficient fine-tuning is faster and matches the prediction performance of full fine-tuning.

2 Related work

Language model personalization has long been recognized as a way to improve user-specific predictions. Early adaptive language modeling approaches included cache models (Kuhn and De Mori, 1990; Jelinek et al., 1991) and Prediction by Partial Matching (PPM) (Cleary and Witten, 1984). Later neural approaches used recurrent neural network language models (RNNLMs) that updated model weights as test text was processed (Mikolov et al., 2010).

Mikolov and Zweig (2012) conditioned an RNNLM on topic information extracted from preceding text. Krause et al. (2018) adapted RNNLMs based on long short-term memory (LSTM) networks during evaluation. The model first scores a text segment, then updates its parameters using gradient descent on that segment before predicting the next segment. These studies examined adaptation to recent text rather than individual-user predictive typing.

For individual-user personalization, Wen et al. (2013) fine-tuned RNNLMs using social-network text from users and their friends, and investigated user-oriented features for speech recognition. King and Cook (2020) compared continued training, interpolation with user-specific models, and recurrent-state priming based on users’ blog posts.

Fowler et al. (2015) studied personalization by simulating touchscreen typing of time-ordered Enron email. They interpolated a background bigram word language model with a user-specific unigram cache. The unigram cache was initialized from earlier emails and updated as typing progressed. Personalization reduced word error rate for noisy touch input compared to the background model alone. Uniform and exponentially decaying caches produced similar results. These caches captured which words a user frequently used, but did not model the contexts in which those words occurred.

Adhikary and Vertanen (2023) extended the study of Fowler et al. (2015) by using character n-gram background models and comparing unigram caching with PPM and a character RNNLM for rescored candidate word predictions. They evaluated typing of Enron email from 44 users, with and without spatial touch noise. They used fixed amounts of text per user for priming and evaluation, comparing primed models with flat-start models that had not observed the seed text. Increasing the background model’s n-gram order im-

proved keystroke savings in both noise conditions, while PPM-based adaptation provided further gains. Combining PPM with the RNNLM achieved the highest keystroke savings in both conditions.

Their RNNLM incorporated user history through hidden-state persistence rather than user-specific parameter updates. Adhikary and Vertanen (2023) observed little benefit from priming the RNNLM and suggested periodically updating neural model parameters on recent writing. We investigate these directions using one-time full-parameter fine-tuning and one-time or progressive LoRA. We also compare progressive conditions with and without seed-set adaptation.

Recent AAC research has explored LLMs for assistance beyond word prediction. KWickChat let users provide a small set of keywords and used an LLM to generate utterances based on those keywords, user profile information, and the conversational partner’s previous turns (Shen et al., 2022). Other work has studied AI-generated phrase suggestions and abbreviation expansion (Valencia et al., 2023; Cai et al., 2024).

For character-level AAC prediction, Gaines and Vertanen (2025) developed a search algorithm for obtaining next-character probabilities from sub-word LLMs. With a domain-adapted OPT-350M model (Zhang et al., 2022), their approach achieved lower average per-character perplexity than the evaluated n-gram, byte-level, and classifier-based alternatives, although inference was slower. Compared with the unadapted model, direct fine-tuning reduced average per-character perplexity and increased average simulated keystroke savings across their test sets. However, their adaptation targeted AAC-like language rather than individual users, and training and evaluation used isolated sentences without context from previous messages.

We extend past work by studying whether a user’s previous text can improve an LLM’s predictions. Specifically, we explore: 1) using previous text in the LLM’s left context, 2) performing one-time parameter adaptation, and 3) performing progressive parameter adaptation.

3 Dataset, procedures, and models

We used the Enron Personalization Validation Set.¹ This dataset contains the sent emails of Enron employees split into 45 development and 44 test users.

¹<https://github.com/google-research-datasets/EnronPersonalizationValidation>

It was assembled by Fowler et al. (2015) to evaluate language model personalization for touchscreen typing. Adhikary and Vertanen (2023) further normalized this dataset to evaluate PPM and recurrent neural network language model (RNNLM) personalization.² We use this latter version.

We split each user’s time-ordered sent emails into two equal-sized sets denoted *seed* (the first 50%) and *evaluation* (the second 50%). The seed set was the only user-specific text available before evaluation. All results were on the evaluation set. This approach follows Adhikary and Vertanen (2023) who evaluated with and without adapting on earlier messages (“primed” versus “flat start”). In the progressive setting, we processed the evaluation set in time-ordered chunks, scoring a chunk before using the chunk to adapt the model.

We used 10 of the 45 development users (denoted DEV) for choosing LLM size, left context length, and fine-tuning hyperparameters. We selected 10 users representative of the distribution over the per-user amount of email over the entire dataset. After fixing all choices, we used 44 test users (denoted TEST) for our final evaluation.

We lowercased all text, excluded phrases containing digits, and retained only the letters a–z, space, and apostrophe. We computed per-character perplexity over this 28-character vocabulary using the algorithm for subword-tokenized LLMs described by Gaines and Vertanen (2025).

We computed keystroke savings as the percentage of keystrokes that a user making no typing mistakes would save by making optimal use of the offered word predictions. We simulated a keyboard with three prediction slots. One slot was reserved for the literal characters typed. Thus, our simulated user could always obtain their desired text. The other two slots extended the currently typed text, searching for the most likely next words under the language model. Our simulator had no explicit word list, rather, we conducted a beam search under the language model looking for sequences ending in a space, comma, exclamation mark, question mark, or period. We assume one keystroke was required to select a prediction and produce any following space. Higher keystroke savings is better.

We tested LLMs from the OPT family (Zhang et al., 2022) as Gaines and Vertanen (2025) found it performed well compared to other model families on conversational style text. As a non-LLM

Language model	Perplexity ↓	Seconds per char
OPT-125M	2.73	0.068
OPT-350M	2.66	0.083
OPT-1.3B	2.57	0.082
OPT-2.7B	2.54	0.092
OPT-6.7B	2.51	0.096
OPT-13B	2.50	0.115

Table 1: Per-character perplexity and inference time on DEV for OPT language models of different sizes.

baseline, we used a 12-gram model with 408 M parameters trained on 21 B characters.³

For approaches that fine-tuned an LLM, we tuned hyperparameters on DEV. For all approaches, we tuned the dropout rate, learning rate, weight decay, and epochs. For LoRA (Hu et al., 2022) approaches, we also tuned adapter rank and alpha. For progressive personalization, we also tuned how many phrases we processed between each model adaptation. See Appendix C for details. The LoRA adapter reduced the average fine-tuning time from 66.9 to 30.8 minutes per user using four NVIDIA A100 GPUs.

4 Experiments

4.1 LLM model size

We first tested OPT LLMs of increasing size. We computed the perplexity of the evaluation split set for each user in the DEV set. The perplexity of each phrase was computed in isolation (i.e. we did not condition on previous phrases in a user’s data). As expected, LLMs with more parameters provided better predictions, though gains started to plateau for larger models (Table 1). Based on this, we decided to use the largest 13-billion-parameter model for our remaining experiments.

4.2 Amount of left context

Next, we explored the impact of how much context to use from a user’s previous emails. We retained recent text from previous phrases and trimmed it from the left to satisfy a specified *left context character limit*, normally removing whole words. The retained context could therefore begin partway through a sentence. As shown in Table 2, increasing the limit from 0 to 512 characters reduced perplexity from 2.50 to 2.05, a relative reduction of approximately 18%. Increasing the limit to 1024 characters provided only a further 1.5% reduction,

²<https://osf.io/45p3j/>

³We used the Dec 2019 huge model from <https://osf.io/ajm7t/>

Left context char	Perplexity ↓	Seconds per char
0	2.50	0.115
64	2.33	0.119
128	2.20	0.125
256	2.11	0.144
512	2.05	0.185
1024	2.02	0.275

Table 2: Per-character perplexity and inference time on DEV with increasing left context.

while inference time increased from 0.185 to 0.275 seconds per character. We therefore chose a left context limit of 512 for the main experiments. Section 4.8 compares context limits of 128, 256, 512, and 1024 characters on TEST for the baseline OPT-13B model without parameter adaptation (denoted Base), one-time LoRA, and seed-initialized progressive LoRA.

4.3 Static models

Our main evaluation results on TEST are shown in Table 3. We tested a static n-gram language model and a pretrained LLM (OPT-13B). As expected, the LLM performed substantially better, reducing perplexity from 3.00 to 2.05 and increasing keystroke savings from 54.8% to 67.5%. The LLM was able to condition on up to 512 characters from previous emails versus only 11 characters for the n-gram model. For comparison, the perplexity on TEST using the LLM using no left context was 2.46. This shows the LLM could leverage large amounts of left context to provide improved user-specific predictions even without user-specific adaptation of model parameters.

4.4 Mixture of n-gram + adaptive PPM

We compared LLM personalization with a statistical baseline model. Our baseline combined a fixed n-gram language model with an adaptive PPM model, similar to the mixture used by [Adhikary and Vertanen \(2023\)](#). PPM is an adaptive variable-order language model originally developed for text compression ([Cleary and Witten, 1984](#)). Similar to an n-gram model, PPM predicts the next symbol from recent context and uses lower-order contexts to smooth its probability estimates. A PPM model of order n conditions on the previous n symbols (in contrast, an n -gram model conditions on the previous $n - 1$ symbols). PPM maintains explicit context-dependent symbol counts that can be updated efficiently during use, making it straightfor-

ward to personalize and update continuously as new text is observed.

We used the character 12-gram model to generate an n-best list of 100 candidate word completions based on a user’s current text context. We then scored the n-best candidates using a user’s PPM model. We used a PPM model of order 11, which could condition on up to the previous 11 characters. For each model, we normalized the candidate probabilities to sum to one over the same n-best list. We then linearly interpolated these probabilities, assigning a weight of 0.75 to the n-gram model and the remaining 0.25 to PPM. We used the highest-ranked candidates under this mixture in our keystroke savings simulation.

For each user, we initialized a new PPM model and primed it on a user’s seed set. During evaluation, n-best scoring did not modify the PPM model. After each target word was committed, we updated PPM once with the committed word and its following space. Thus, the n-gram component of the mixture model remained static, whereas the PPM component used seed-set personalization followed by progressive adaptation on the evaluation set.

As shown in Table 3, compared to the static n-gram model, the mixture model reduced perplexity from 3.00 to 2.88. Keystroke savings also improved from 54.8% to 56.7%. However, despite adapting to a user’s text, the mixture model still performed substantially worse than OPT-13B without parameter adaptation. When conditioned on a user’s previous sent emails, OPT-13B achieved a perplexity of 2.05 and keystroke savings of 67.5%.

4.5 One-time personalization

We first tried full parameter fine-tuning of the LLM on each user’s seed set. While this only reduced the perplexity slightly, from 2.05 to 2.03, it did increase the keystroke savings by 0.5%. We found a LoRA adapter performed similar to full fine-tuning with the same perplexity of 2.03 with a slight decrease in keystroke savings from 68.0% to 67.9%.

4.6 Progressive personalization

We attempted to further improve the LoRA adapter learned from the seed data by progressively fine-tuning the adapter every 128 phrases during evaluation. While this slightly reduced perplexity from 2.03 to 2.02, it provided only a small improvement in keystroke savings.

To separate the contribution of progressive updates from seed-set adaptation, we also evaluated

Language model	Adaptation		Perplexity ↓	Keystroke savings ↑
	method	on seed? on eval?		
12-gram	none	no no	3.00 ± 0.047	$54.8 \pm 0.5\%$
12-gram + PPM-11	PPM	yes yes	2.88 ± 0.050	$56.7 \pm 0.5\%$
OPT-13B	none	no no	2.05 ± 0.038	$67.5 \pm 0.6\%$
OPT-13B	one-time full fine-tuning	yes no	2.03 ± 0.039	$68.0 \pm 0.6\%$
OPT-13B	one-time LoRA	yes no	2.03 ± 0.038	$67.9 \pm 0.6\%$
OPT-13B	one-time LoRA + progressive LoRA	yes yes	2.02 ± 0.039	$67.9 \pm 0.6\%$
OPT-13B	progressive LoRA(base-initialized)	no yes	2.03 ± 0.039	$67.8 \pm 0.6\%$
OPT-13B	cumulative-reset LoRA (base-initialized)	no yes	2.03 ± 0.040	$67.8 \pm 0.6\%$

Table 3: Per-character perplexity and keystroke savings of different language models and adaptation approaches. Results are means over the 44 TEST users. $\pm$ denotes 95% nonparametric user-level bootstrap confidence intervals and is the larger distance from the mean to either bootstrap endpoint. OPT-13B results conditioned on up to 512 characters of previous context. In the final row, each non-final block of 128 evaluation phrases was scored before being used for further LoRA fine-tuning.

base-initialized progressive personalization. In this approach, we started from the non-personalized OPT-13B checkpoint and did not adapt on the seed set. As with the normal progressive method, we updated the LoRA adapter after every 128 evaluation phrases. This did only slightly worse with a perplexity of 2.03 and a keystroke savings of 67.8%. This shows it is possible to learn a useful LoRA adapter even without a large amount of initial text.

4.7 Cumulative progressive personalization

We also evaluated a variant of the progressive method in which we periodically relearned the LoRA adapter from the cumulative set of all evaluation text seen thus far. The hope was that this would allow more robust updates to the adapter as the amount of text increased.

As in the base-initialized progressive condition described in the previous subsection, we did not adapt on the seed set, and the evaluation set was processed chronologically in 128-phrase chunks. The cumulative variant differed in how the adapter was updated. After scoring each non-final chunk, we reloaded the original non-personalized OPT-13B checkpoint, initialized a new LoRA adapter, and fine-tuned it on all evaluation phrases observed up to that point. In contrast, the base-initialized progressive method continued fine-tuning the existing adapter on only the newly scored chunk. Thus, the cumulative adaptation sets contained 128 phrases, 256 phrases, 384 phrases, and so on.

The cumulative approach obtained a mean per-character perplexity of 2.03 and mean keystroke savings of 67.84% on TEST. These mean results were similar to those of base-initialized sequential progressive LoRA. Compared with seed-initialized

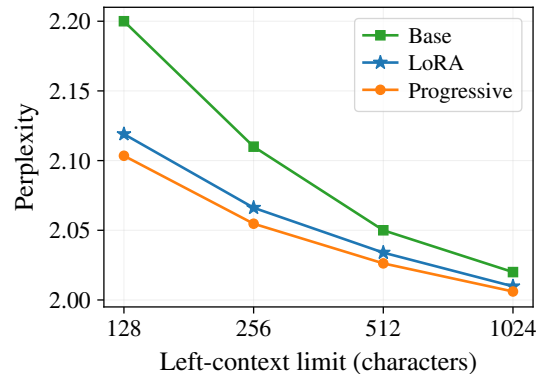


Figure 1: Perplexity at left-context limits of 128, 256, 512 and 1024 characters. Results on TEST for OPT-13B without fine-tuning (Base), personalized using a LoRA adapter on the seed set (LoRA), and personalized using a LoRA adapter on the seed set and updated during evaluation (Progressive).

progressive LoRA, the cumulative approach had higher mean perplexity and lower mean keystroke savings. The cumulative approach was also slower. Base-initialized progressive LoRA required an average of 38 seconds per fine-tuning update, while the cumulative approach needed 233 seconds.

4.8 Left context influence

Despite trying a variety of parameter fine-tuning approaches, we saw little gain beyond simply using the pretrained OPT-13B model conditioned on 512 characters. To investigate whether long left context was really limiting the upside to model parameter fine-tuning, we compared per-character perplexity for Base, one-time LoRA, and one-time LoRA + progressive LoRA at left context limits of 128, 256, 512, and 1024 characters. Base used pretrained OPT-13B without parameter adaptation at

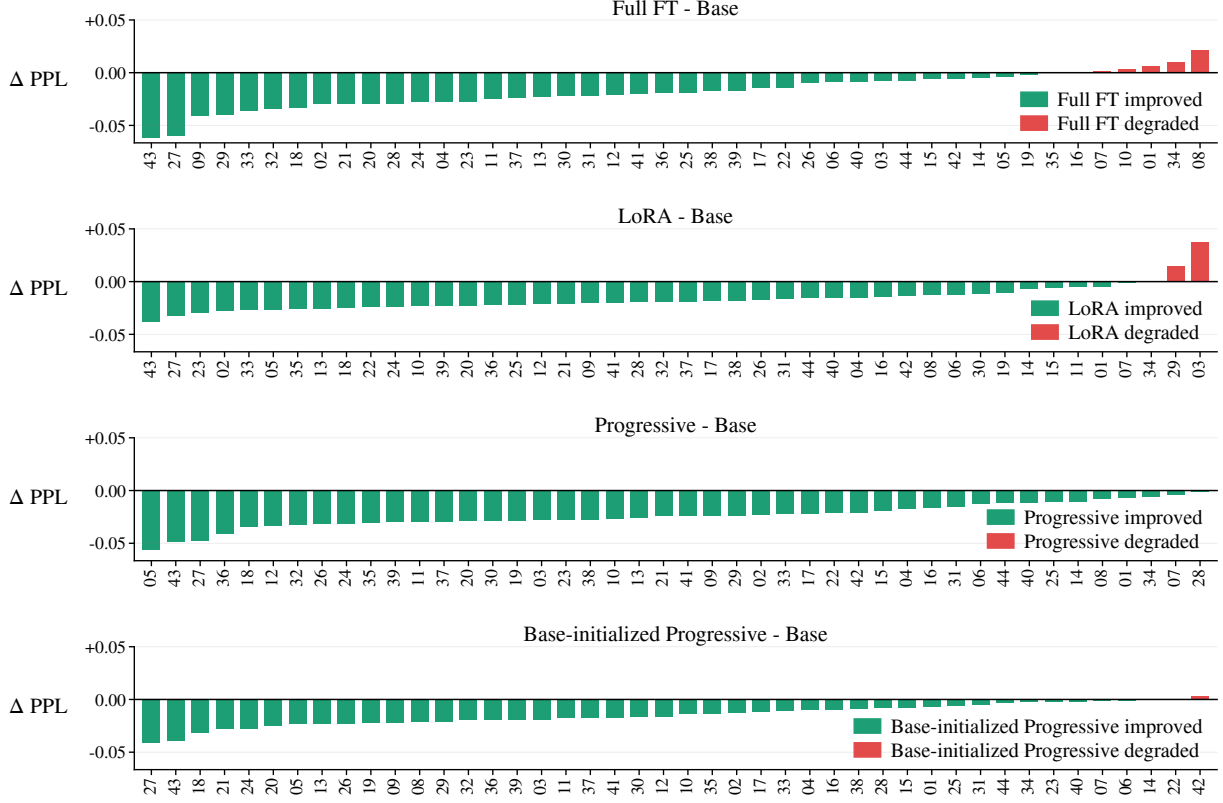


Figure 2: Per-user changes in per-character perplexity (PPL) relative to the non-personalized OPT-13B baseline (Base) for the 44 TEST users. From top to bottom, the panels show full-parameter fine-tuning, one-time LoRA, seed-initialized progressive LoRA, and base-initialized progressive LoRA. ΔPPL denotes the method’s perplexity minus that of Base; negative values indicate lower perplexity. Users are sorted independently from the largest to the smallest improvement in each panel.

each context limit. The left-context limit controlled how much text from previous phrases was supplied during prediction, not the length of the training examples. For each user, one-time LoRA used the same seed-trained adapter at all four context limits, without further fine-tuning.

For progressive LoRA, we ran the chronological predict-then-update procedure separately at 256 and 512 characters, using the same fine-tuning settings and 128-phrase update interval. At 128 and 1024 characters, we re-evaluated the saved adapter sequence from the 512-character run without repeating the training. The first chunk used the seed-trained adapter, and each later chunk used an adapter updated only on earlier chunks. Thus, no chunk was scored using an adapter trained on that chunk. As shown in Figure 1, perplexity decreased as left context increased. Both personalized models had lower mean perplexity than the non-personalized model at every tested left-context limit, and progressive LoRA had the lowest perplexity at all four context limits. At 1024 characters, mean perplexity was approximately 2.02

for Base, 2.01 for one-time LoRA, and 2.00 for progressive LoRA. Notably, with less left context, the perplexity advantage of the approaches that adapted model parameters increased. This suggests that parameter-based personalization may still be useful when shorter left context is desired to reduce inference costs or avoid retaining a user’s past writing between text entry sessions.

4.9 Per-user variability

We also inspected per-user changes in perplexity relative to the non-personalized OPT-13B baseline. Full fine-tuning and LoRA reduced perplexity for 38 and 41 of the 44 users, respectively. Progressive LoRA reduced perplexity for all users, while base-initialized progressive LoRA reduced perplexity for 43 of 44 users, with only a small increase for one user (Figure 2).

Thus, the improvements we saw were broadly distributed across users rather than being driven by a small number of users with unusually large gains. The base-initialized result also shows that chronological updates can provide broadly consistent per-

sonalization gains even without adaptation on the seed half, although progressive LoRA produced the most consistent per-user improvements. Figure 4 in Appendix B shows per-user keystroke savings for Base and the three seed-adapted conditions. Keystroke savings varied across users, whereas differences among these methods for a given user were generally small.

4.10 Paired comparisons between methods

We compared Base, one-time LoRA, one-time LoRA + progressive LoRA, and progressive LoRA (base-initialized). The one-time LoRA + progressive LoRA condition started from a seed-trained adapter, whereas progressive LoRA (base-initialized) started from Base without seed-set adaptation. These comparisons focused on the contributions of seed-set adaptation and progressive updates within LoRA, using Base as the reference. We evaluated all six method pairs for perplexity and keystroke savings, using a left context character limit of 512. Comparisons with full fine-tuning, the statistical baselines, and cumulative-reset personalization were outside the scope of this analysis.

For each method pair and metric, we calculated the difference between method A and method B for each of the TEST users, subtracting B from A. We report the mean of these 44 differences. To estimate a 95% percentile bootstrap confidence interval, we generated 100,000 bootstrap samples. Each sample contained 44 differences drawn with replacement from the original 44 user differences. We calculated the mean of each sample and used the 2.5th and 97.5th percentiles of these bootstrap means as the interval endpoints (Efron and Tibshirani, 1986).

The mean differences and bootstrap confidence intervals describe the size of the differences between methods and the uncertainty in these estimates. We used two-sided Wilcoxon signed-rank tests to assess the statistical significance of differences between methods (Wilcoxon, 1945). We calculated exact conditional p -values, excluding zero differences and assigning average ranks to tied absolute differences. We report W as the smaller of the rank sums for positive and negative differences, and n as the number of nonzero paired differences.

We applied Holm’s step-down Bonferroni correction across all 12 signed-rank tests (six method pairs for each of the two metrics) to control the family-wise error rate at 0.05 (Holm, 1979). Statistical significance was determined using the Holm-adjusted p -values, not the bootstrap intervals. The

bootstrap intervals are unadjusted and should be interpreted individually, not as providing simultaneous 95% coverage across all comparisons.

The signed-rank test assumes independent user differences that are symmetric about zero under the null hypothesis. It is not specifically a test of the mean difference.

Table 4 reports the paired mean differences, confidence intervals, comparison results, and per-user improvement counts. 11 of the 12 signed-rank comparisons were significant after Holm correction. The unadjusted bootstrap intervals showed a consistent pattern: they excluded zero for the 11 statistically significant comparisons and included zero for the remaining non-significant comparison.

All three adaptation conditions had lower mean perplexity and higher mean keystroke savings than Base. One-time LoRA + progressive LoRA achieved the lowest mean perplexity and highest mean keystroke savings among the four conditions. Relative to one-time LoRA, it reduced mean perplexity by 0.00772 and increased mean keystroke savings by 0.103%, improving perplexity for 34 users and keystroke savings for 36. Both comparisons were statistically significant, but the additional typing-effort benefit was small.

Progressive LoRA (base-initialized) also improved over Base without seed-set adaptation. Among the 12 comparisons at the 512-character context setting, the only nonsignificant result was its perplexity difference from one-time LoRA (exact conditional two-sided Wilcoxon signed-rank test: $W = 349$, $n = 44$, Holm-adjusted $p = 0.0896$). Its mean keystroke savings was 0.067% lower than one-time LoRA ($W = 285$, $n = 43$, Holm-adjusted $p = 0.0450$). All ten other comparisons had Holm-adjusted $p < 0.001$. Overall, the paired analysis supports small additional benefits from parameter adaptation beyond recent user-specific context, rather than large improvements in typing effort. All statistics were computed from unrounded per-user scores.

5 Discussion

Our results separate the benefit of using a stronger language model from the additional benefit of per-user adaptation. The mixture of an n -gram model with adaptive PPM improved both perplexity and keystroke savings relative to the static 12-gram model. This shows that a lightweight count-based model can benefit from adapting to a user’s text.

Metric	Method A	Method B	Mean Δ	95% CI	Result (Holm)	Users: A vs. B		
						Better	Worse	Tied
PPL↓	LoRA	Base	-0.016	[-0.020, -0.012]	A better	41	3	0
PPL↓	Progressive	Base	-0.024	[-0.028, -0.021]	A better	44	0	0
PPL↓	Progressive (base)	Base	-0.015	[-0.018, -0.012]	A better	43	1	0
PPL↓	Progressive	LoRA	-0.008	[-0.012, -0.004]	A better	34	10	0
PPL↓	Progressive (base)	LoRA	+0.002	[-0.003, +0.005]	Not sig.	19	25	0
PPL↓	Progressive	Progressive (base)	-0.009	[-0.012, -0.007]	A better	41	3	0
KS↑	LoRA	Base	+0.313	[+0.269, +0.357]	A better	43	1	0
KS↑	Progressive	Base	+0.416	[+0.363, +0.468]	A better	42	1	1
KS↑	Progressive (base)	Base	+0.246	[+0.197, +0.297]	A better	42	1	1
KS↑	Progressive	LoRA	+0.103	[+0.072, +0.135]	A better	36	8	0
KS↑	Progressive (base)	LoRA	-0.067	[-0.115, -0.021]	B better	18	25	1
KS↑	Progressive	Progressive (base)	+0.170	[+0.134, +0.209]	A better	42	0	2

Table 4: Paired comparisons of per-character perplexity (PPL) and keystroke savings (KS) over TEST users with a 512-character left-context. LoRA denotes one-time adaptation; Progressive and Progressive (base) denote seed- and base-initialized progressive LoRA. Differences are A minus B: negative PPL and positive KS differences favor A. KS differences are percentage points. CIs are unadjusted 95% paired-bootstrap intervals. The result identifies the method with the better observed mean for significant comparisons (two-sided Wilcoxon signed-rank test, Holm-adjusted $p < 0.05$ across 12 tests). “Not sig.” denotes a nonsignificant comparison.

However, the adaptive mixture was still substantially worse than the non-personalized OPT-13B baseline. Thus, in our setting, a pretrained LLM using long recent context provided a larger improvement than adapting the statistical language model. Since OPT-13B also used a much longer left context than the 12-gram model, this comparison reflects both the stronger pretrained model and its ability to condition on more of a user’s recent text.

The context-length comparison in Figure 1 shows how left context and model parameter adaptation interact. Perplexity decreased as the left-context limit increased for the baseline model, the LoRA-adapted model, and the progressively adapted LoRA model. The progressive LoRA method had the lowest mean perplexity at 128, 256, 512, and 1024 characters of left context, but the gap between the Base model and both adapted models narrowed as more context became available. The advantage of the progressive LoRA method over one-time LoRA adaptation also became smaller at the longer context limits, with the additional perplexity reduction halving between 512 and 1024 characters. This suggests that left context and parameter adaptation provide partly overlapping user-specific information. Adaptation was more useful when less recent text was available to the model, whereas long left context allowed the non-personalized model to recover more of this information without changing its parameters.

LoRA achieved predictive performance comparable to full-parameter fine-tuning while requiring less fine-tuning time. The progressive LoRA method, which updated the adapter every 128 phrases, further reduced mean perplexity and improved all 44 TEST users. However, the additional improvement in mean keystroke savings over one-time LoRA adaptation was small. Perplexity measures the probability assigned across all characters, whereas keystroke savings depend on whether the desired word appears in one of the available prediction slots. Small changes in the character probability distribution may therefore reduce perplexity without changing the word completions offered.

The base-initialized progressive model also improved over the non-personalized OPT-13B baseline and reduced perplexity for 43 of 44 users, despite receiving no adaptation on the seed set. This shows that chronological updates can provide useful personalization when earlier user data are not available. In contrast, cumulative progressive personalization did not improve over the simpler progressive procedure. Reinitializing the adapter and relearning it from all evaluation text observed thus far provided no additional benefit. Although the mean differences among the personalized OPT-13B models were small, seed-initialized progressive LoRA reduced perplexity for 34 of the 44 users compared with one-time LoRA. Compared with Base, it reduced perplexity for all 44 users.

6 Conclusion

We evaluated non-personalized, one-time personalized, and progressively personalized language models for predictive typing under a chronological predict-then-update protocol. A non-personalized OPT-13B model using long left context already outperformed both the static 12-gram model and the adaptive 12-gram + PPM mixture. Parameter adaptation provided smaller additional gains: LoRA closely matched full-parameter fine-tuning, while progressive LoRA obtained the lowest mean perplexity and the most consistent per-user improvements. Its relative advantage was larger when the available left context was shorter.

We also found that periodically updating the pretrained model using a user’s new messages improved predictions, even without first fine-tuning it on a collection of that user’s earlier messages. The limited change in keystroke savings among the adapted OPT-13B models showed that lower perplexity does not necessarily produce a comparable reduction in simulated typing effort. This is likely because small changes in a model’s predictive ability (as measured by perplexity), may not materially impact a keyboard’s top word predictions. Taken together, these results identify long recent user-specific context as the primary source of improvement, with efficient progressive adaptation providing a smaller, complementary benefit.

Limitations

Our work has a number of limitations. We used one global set of hyperparameters selected on development users and applied it to all TEST users, even though users had very different amounts of available text. Users with short, medium, and long histories may benefit from different fine-tuning hyperparameters, such as learning rate, number of epochs, and dropout.

We evaluated on a dataset containing sent emails of Enron employees. This data is now quite dated and may not reflect modern language usage. The genre of business email may also not be representative of other predictive typing settings such as informal person-to-person conversations that might occur on mobile devices, social media, or be needed in the everyday lives of AAC users. However, to our knowledge, there are no similar datasets containing the communications of a large number of people over a long period of time.

Our progressive adaptation method used an up-

date frequency of 128 phrases based on experiments we conducted on development data. But we did not study the trade-off between update frequency and computational cost in a deployed system. It is possible a different chunk size might offer a more acceptable end-user experience.

The LLM we tested had 13 B parameters and required a powerful GPU for adaptation and inference. The strong ability of left context to provide user personalization likely benefited from the “emergent” in-context learning abilities of the large model we used. Deploying such a large model on a user’s device, as might be necessitated to maintain user privacy, may require advances in mobile device hardware (e.g. GPU resources) and the software infrastructure to efficiently deploy powerful models on edge devices (e.g. model quantization). Alternatively, it would be interesting to investigate combining a large cloud-based model with an on-device personalized model.

We used an OPT LLM released in 2022. While past work showed the OPT family performed well compared to a variety of other model families (Gaines and Vertanen, 2025), there may be more recent LLMs that perform even better. While OPT performed well in English for our experiments, it may not be the best choice for users who need to write in other languages.

Our findings were based on simulating the keystroke savings of a hypothetical perfect user. This user made no typing mistakes and made optimal use of offered word predictions. Actual real-world user performance may differ and needs validation by observing users writing a substantial amount of text over a long period of time.

We did not systematically repeat the fine-tuning experiments with different random seeds. Our paired-bootstrap confidence intervals reflect uncertainty from sampling users, not variability across training runs. Repeating fine-tuning with different seeds would help assess the stability of the adaptation gains we saw in our experiments.

Acknowledgments

This material is based upon work supported by the NSF under Grant No. IIS-2402876.

References

Jiban Adhikary and Keith Vertanen. 2023. [Language model personalization for improved touchscreen typing](#). In *Interspeech 2023*, pages 1344–1348.

- David R. Beukelman and Janice C. Light. 2020. *Augmentative and Alternative Communication: Supporting Children and Adults with Complex Communication Needs*, 5th edition. Paul H. Brookes Publishing.
- Shanqing Cai, Subhashini Venugopalan, Katie Seaver, Xiang Xiao, Katrin Tomanek, Sri Jalasutram, Meredith Ringel Morris, Shaun K. Kane, Ajit Narayanan, Robert L. MacDonald, Emily Kornman, Daniel Vance, Blair Casey, Steve M. Gleason, Philip Q. Nelson, and Michael P. Brenner. 2024. [Using large language models to accelerate communication for eye gaze typing users with ALS](#). *Nature Communications*, 15(1):9449.
- John G. Cleary and Ian H. Witten. 1984. [Data compression using adaptive coding and partial string matching](#). *IEEE Transactions on Communications*, 32(4):396–402.
- Bradley Efron and Robert Tibshirani. 1986. [Bootstrap methods for standard errors, confidence intervals, and other measures of statistical accuracy](#). *Statistical Science*, 1(1):54–75.
- Andrew Fowler, Kurt Partridge, Ciprian Chelba, Xiaojun Bi, Tom Ouyang, and Shumin Zhai. 2015. [Effects of language modeling and its personalization on touchscreen typing performance](#). In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*, CHI ’15, pages 649–658. ACM.
- Dylan Gaines and Keith Vertanen. 2025. [Adapting large language models for character-based augmentative and alternative communication](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 15273–15291, Suzhou, China. Association for Computational Linguistics.
- Lisa Haake, Sebastian Wallot, Monika Tschense, and Joachim Grabowski. 2024. [Global temporal typing patterns in foreign language writing: Exploring language proficiency through recurrence quantification analysis \(RQA\)](#). *Reading and Writing*, 37(2):385–417.
- Sture Holm. 1979. [A simple sequentially rejective multiple test procedure](#). *Scandinavian Journal of Statistics*, 6(2):65–70.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. [LoRA: Low-rank adaptation of large language models](#). In *International Conference on Learning Representations*.
- F. Jelinek, B. Meriello, S. Roukos, and M. Strauss. 1991. [A dynamic language model for speech recognition](#). In *Speech and Natural Language: Proceedings of a Workshop Held at Pacific Grove, California, February 19–22, 1991*, pages 293–295.
- Milton King and Paul Cook. 2020. [Evaluating approaches to personalizing language models](#). In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 2461–2469, Marseille, France. European Language Resources Association.
- Bryan Klimt and Yiming Yang. 2004. [The Enron corpus: A new dataset for email classification research](#). In *Machine Learning: ECML 2004*, volume 3201 of *Lecture Notes in Computer Science*, pages 217–226, Berlin, Heidelberg. Springer.
- Ben Krause, Emmanuel Kahembwe, Iain Murray, and Steve Renals. 2018. [Dynamic evaluation of neural sequence models](#). In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 2766–2775. PMLR.
- Roland Kuhn and Renato De Mori. 1990. [A cache-based natural language model for speech recognition](#). *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 12(6):570–583.
- Tomáš Mikolov, Martin Karafiát, Lukáš Burget, Jan Černocký, and Sanjeev Khudanpur. 2010. [Recurrent neural network based language model](#). In *Interspeech 2010*, pages 1045–1048.
- Tomáš Mikolov and Geoffrey Zweig. 2012. [Context dependent recurrent neural network language model](#). In *2012 IEEE Spoken Language Technology Workshop (SLT)*, pages 234–239. IEEE.
- Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. [ZeRO: Memory optimizations toward training trillion parameter models](#). In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16. IEEE.
- Zhanna Sarsenbayeva, Niels van Berkel, Chu Luo, Vasilis Kostakos, and Jorge Goncalves. 2017. [Challenges of situational impairments during interaction with mobile devices](#). In *Proceedings of the 29th Australian Conference on Computer-Human Interaction*, OzCHI ’17, pages 477–481. Association for Computing Machinery.
- Junxiao Shen, Boyin Yang, John J. Dudley, and Per Ola Kristensson. 2022. [KWickChat: A multi-turn dialogue system for AAC using context-aware sentence generation by bag-of-keywords](#). In *Proceedings of the 27th International Conference on Intelligent User Interfaces*, IUI ’22, pages 853–867. Association for Computing Machinery.
- Keith Trnka and Kathleen McCoy. 2008. [Evaluating word prediction: Framing keystroke savings](#). In *Proceedings of ACL-08: HLT, Short Papers*, pages 261–264, Columbus, Ohio. Association for Computational Linguistics.
- Stephanie Valencia, Richard Cave, Krystal Kallarackal, Katie Seaver, Michael Terry, and Shaun K. Kane. 2023. [“The Less I Type, the Better”: How AI Language Models Can Enhance or Impede Communication for AAC Users](#). In *Proceedings of the 2023 CHI Conference on Human Factors in Computing*

Systems, CHI '23, pages 1–14. Association for Computing Machinery.

Tsung-Hsien Wen, Aaron Heidel, Hung-yi Lee, Yu Tsao, and Lin-shan Lee. 2013. [Recurrent neural network based language model personalization by social network crowdsourcing](#). In *Interspeech 2013*, pages 2703–2707.

Frank Wilcoxon. 1945. [Individual comparisons by ranking methods](#). *Biometrics Bulletin*, 1(6):80–83.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. [OPT: Open pre-trained transformer language models](#). *arXiv preprint arXiv:2205.01068*.

A Enron dataset processing

We started with the version of the original Enron Personalization Validation Set provided by [Adhikary and Vertanen \(2023\)](#). For each Enron employee, the dataset has a time-ordered list of phrases from the employee’s sent emails. We kept only phrases that contained no digits. Each retained phrase was then converted to lowercase and filtered so that only letters, spaces, and apostrophes remained. Table 5 summarizes the total amount of data in the full DEV and TEST splits after our preprocessing. As shown in Table 5, the amount of text ranged widely between users. We split each user’s chronological stream in half, using the first half as seed data and the second half for evaluation.

Figure 3 shows the number of phrases remaining after preprocessing for each user in the full development and test partitions. To reduce the time needed for hyperparameter search, we selected DEV users from our full development set: dev03, dev06, dev09, dev14, dev21, dev24, dev27, dev34, dev36, and dev37. We used this subset for model selection and hyperparameter tuning. Our TEST set included all 44 users in the test partition.

Partition	Total			Per-user words		
	Users	Phrases	Words	Median	Min.	Max.
Development	45	109,696	1,069,532	13,585	2,581	177,216
Test	44	92,784	968,531	11,651	2,196	107,709

Table 5: Dataset statistics after preprocessing.

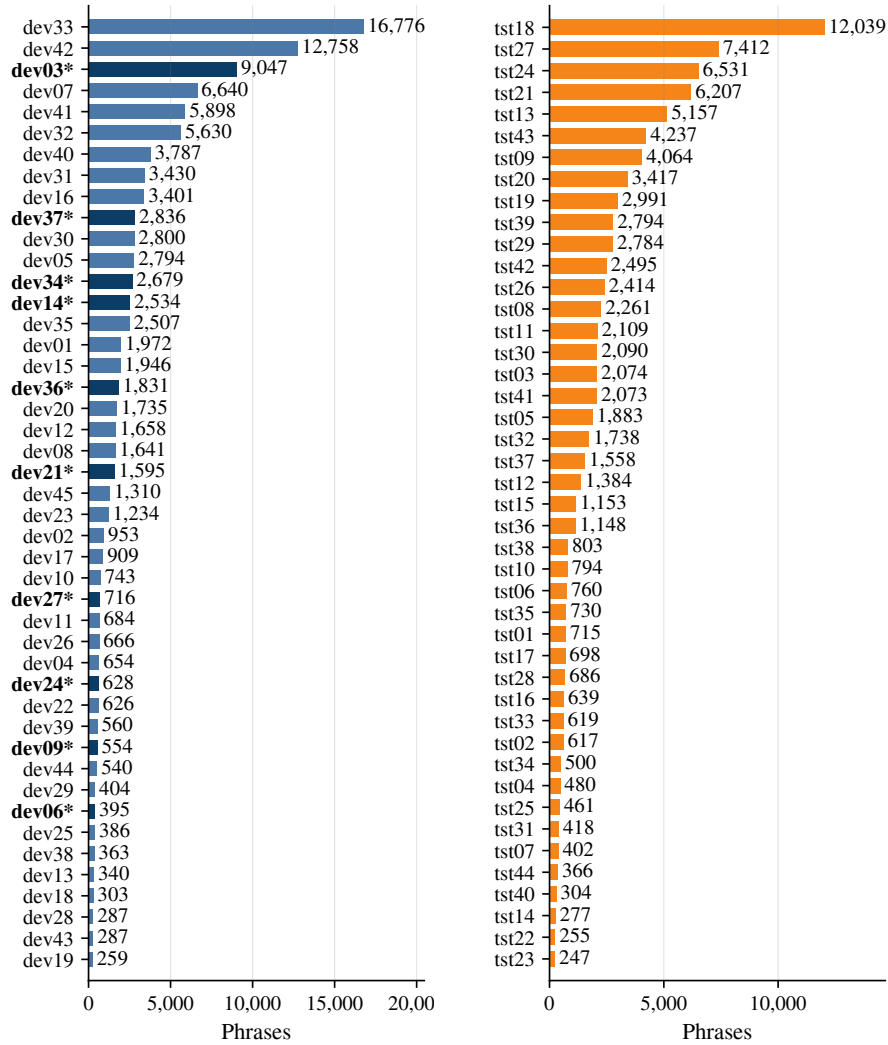


Figure 3: Number of phrases available after preprocessing for each user in the full development partition (left) and the TEST set (right). Users are ordered from most to fewest phrases. Asterisks and darker labels in the left panel mark the 10 users selected for the DEV set used for model selection and hyperparameter tuning.

B Per-user keystroke savings

Figure 4 shows per-user keystroke savings for the non-personalized OPT-13B baseline and the three seed-adapted conditions. Keystroke savings varied substantially across users, whereas differences among the methods for a given user were generally small.

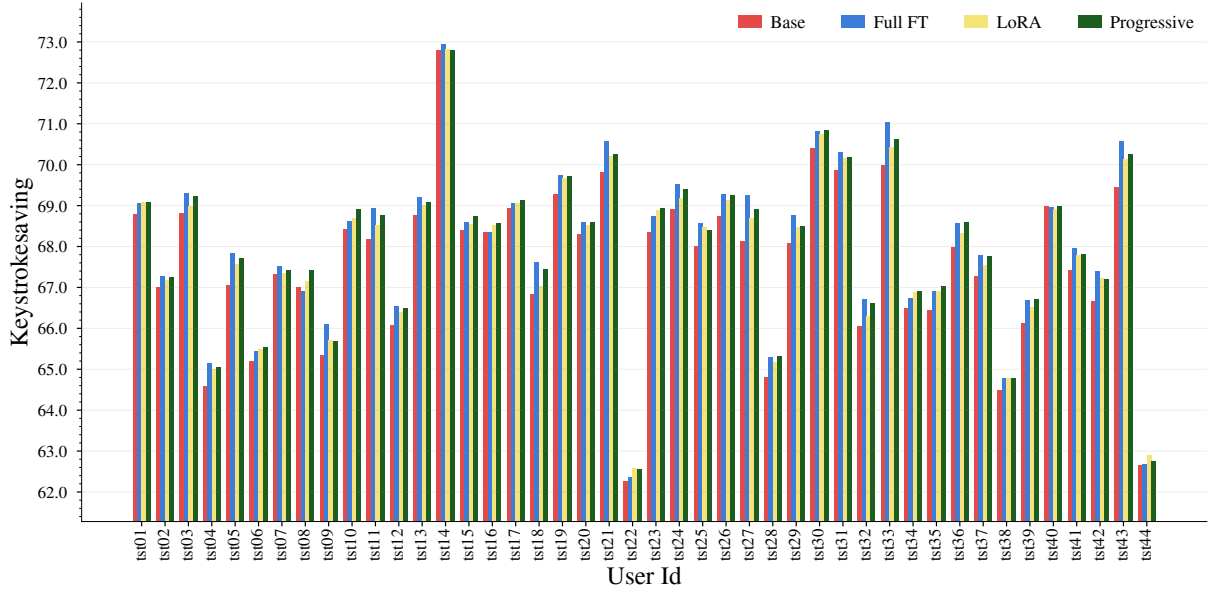


Figure 4: Per-user keystroke savings on TEST for the Base, full-parameter fine-tuning, LoRA fine-tuning, and progressive LoRA (Progressive). Higher values indicate a greater percentage reduction in keystrokes relative to letter-by-letter typing. Users are shown in test-user ID order.

C Hyperparameter tuning

We tuned hyperparameters for full fine-tuning, one-time LoRA, and seed-initialized progressive LoRA. For each method, we selected the settings that gave the lowest mean per-character perplexity across our DEV set, drawn from the full partition of 45 development users. Each tuning trial ran on four NVIDIA A100 GPUs with 40 GB of memory each. We used DeepSpeed ZeRO stage 3 (Rajbhandari et al., 2020) to reduce memory requirements on each GPU. All trials used OPT-13B with up to 512 characters of left context. Hyperparameter tuning used approximately two GPU-days per method. We measured LLM inference latency using FP16 on a single NVIDIA A100 GPU per evaluation process.

We varied the dropout rate, learning rate, weight decay, and the number of training epochs. We selected the learning rate from the candidate set [5e-6, 7e-6, 8e-6, 1e-5, 1.3e-5, 1e-4, 3e-4, 3e-3]. The learning-rate schedule had two additional tunable parameters: the number of warmup steps, selected from 0 to 10, and the stable fraction, selected from 0.00 to 0.80. The stable fraction controlled the fraction of training steps for which the learning rate was held fixed before decay.

The progressive LoRA method had a parameter *chunk size* controlling how often we re-fine-tuned the adapter. We started each of DEV users with a

model fine-tuned on their seed set using the best hyperparameters found for the non-progressive LoRA method. We then selected the chunk size from the candidate set [64, 128, 256, 512, and 1024]. Table 6 shows the values of our tuned hyperparameters for each of our methods.

Adaptation	Chunk size	LoRA rank	LoRA α	Dropout	LR schedule	Warmup steps	Stable fraction	Learning rate	Weight decay	Epochs
Full fine-tuning	–	–	–	0.20	WSD	2	0.15	1×10^{-5}	0.05	2
One-time LoRA	–	8	16	0.10	WSD	2	0.15	3×10^{-4}	0.00	2
Progressive LoRA (seed-initialized)	128	8	16	0.10	WSD	2	0.15	3×10^{-4}	0.00	1
Progressive LoRA (base-initialized)	128	8	16	0.10/0.05 ^a	Linear	2	–	3×10^{-4}	0.00	1
Cumulative progressive LoRA	128	8	16	0.10	Linear	2	–	3×10^{-4}	0.00	1

^aAdapter dropout was 0.10 for the first update and 0.05 for subsequent updates.

Table 6: Hyperparameters used for the five OPT-13B personalization methods. Chunk size is measured in phrases. For progressive methods, entries describe evaluation-stage updates and epochs are per update. LR denotes learning rate; WSD denotes warmup–stable–decay, and Linear denotes linear decay after warmup. Dropout applies to the model for full fine-tuning and to the adapter otherwise. Dashes indicate inapplicable settings.